

RAPPORT PROJET ALGORITHME AVANCÉ

Sujet 7: “Liste de priorité LIFO”

Groupe composant: -ANDRINIAINA Henintsoa Anthonny(389)
-RANDRIANOELISON Jerinirina Fiderana(391)
-RASAMIMANANA Heritiana Hiarilala(399)

Domaine d'application : Historique de navigation Internet (Bouton Retour)

Introduction:

Dans le cadre de notre projet, l'objectif est de concevoir une structure de données LIFO personnalisée. Afin d'illustrer concrètement ce concept abstrait, nous avons développé un **programme de simulation d'historique de navigation Internet**. Chaque fois qu'un utilisateur visite une nouvelle adresse URL, celle-ci est empilée au sommet de l'historique. Lorsqu'il clique sur le bouton "Retour", la dernière page consultée est immédiatement retirée pour restituer la page précédente.

Répartition des Tâches:

Pour la langage de programmation nous avons choisis la language Python.
Maintenant voyons les codes avec leurs explication:

L'entête du code est la brique de base de notre structure. Comme les bibliothèques de structures de données sont interdites, cette classe crée notre propre type d'objet en mémoire.

-La classe **Noeud** :

```
main.py / PileLIFO / afficher_historique
1 class Noeud:
2     def __init__(self, donnee):
3         self.donnee = donnee
4         self.suivant = None
5
```

Pour l'entête du code est la brique de base de notre structure. Comme les bibliothèques de structures de données sont interdites, cette classe crée notre propre type d'objet en mémoire.

"self.donnee = donnee" : Cet attribut stocke l'information brute, c'est-à-dire l'adresse URL du site web visité

"self.suivant = None" : C'est la référence

-La structure **PileLIFO** et son initialisation:

```

6 class PileLIFO:
7     def __init__(self):
8         self.sommet = None
9

```

Ce code là est la classe maîtresse qui contrôle le pile LIFO;

“**self.sommet = None**” : Cet attribut est crucial. Il pointe toujours sur le maillon qui se trouve tout en haut de la pile. Si la pile est vide (au démarrage du programme), le sommet vaut None.

-L"insertion visiter_page:

```

9
10 def visiter_page(self, nouvelle_url):
11     nouveau_noeud = Noeud(nouvelle_url)
12     nouveau_noeud.suivant = self.sommet
13     self.sommet = nouveau_noeud
14     print(f"Vous visitez : {nouvelle_url}")
15

```

Ce code est une méthode permet d'ajouter un site web au sommet de l'historique, qui respecte la règle LIFO en 3 étapes:

1. On crée un nouvel objet Noeud contenant la nouvelle URL.
2. **nouveau_noeud.suivant = self.sommet** : Le nouveau nœud se lie à l'ancien sommet. Il se place "au-dessus" de la page précédente.
3. **self.sommet = nouveau_noeud** : On met à jour la référence de la pile : le nouveau nœud devient officiellement le nouveau sommet.

- La suppression bouton_retour:

```

15
16 def bouton_retour(self):
17     if self.sommet is None:
18         print("Aucun historique, vous êtes déjà sur la page d'accueil.")
19         return None
20
21     page_marquee = self.sommet.donnee
22     self.sommet = self.sommet.suivant
23     print(f"Retour : vous quittez {page_marquee}")
24     return page_marquee
25

```

Ce code est une méthode qui simule le clic sur le bouton "Retour" du navigateur. Elle retire l'élément au sommet.

Sécurité : Elle vérifie d'abord si la pile est vide “**if self.sommet is None**” pour éviter que le programme ne plante.

Sauvegarde : Elle extrait l'URL du sommet actuel dans “`page_marquee`” pour pouvoir l'afficher et la renvoyer.

Mise à jour : “`self.sommet = self.sommet.suivant`” : Le sommet recule d'un cran en pointant sur le nœud du dessous. L'ancienne page est ainsi déconnectée de la chaîne et supprimée de la mémoire.

-Le parcours de la structure afficher_historique :

```
26     def afficher_historique(self):|
27         if self.sommet is None:
28             print("\nHistorique vide")
29             return
30
31         print("\nHistorique de navigation :")
32         actuel = self.sommet
33         while actuel is not None:
34             print(f"- {actuel.donnee}")
35             actuel = actuel.suivant
36         print()
```

Ce code permet de visualiser l'état de la pile,

On a utilisé une variable temporaire “`actuel`” initialisée au “`sommet`”, la boucle “`while actuel is not None`” est la parcourt de la liste chaînée de maillon en maillon (`actuel = actuel.suivant`) en affichant chaque URL avec un tiret (-). Elle s'arrête proprement dès qu'elle atteint le bout de la chaîne (None).

-L'interface utilisateur: le bloc principal

```

38  if __name__ == "__main__":
39      nav = PileLIFO()
40      print("Bienvenue sur le navigateur LIFO")
41
42      while True:
43          print("\n1. Visiter une page")
44          print("2. Retour")
45          print("3. Historique")
46          print("4. Quitter")
47
48          choix = input("Choix : ")
49
50          if choix == "1":
51              url = input("Adresse du site : ")
52              nav.visiter_page(url)
53          elif choix == "2":
54              nav.bouton_retour()
55          elif choix == "3":
56              nav.afficher_historique()
57          elif choix == "4":
58              print("Fermeture du programme.")
59              break
60          else:
61              print("Choix invalide.")

```

Ce code est le contrôleur de notre application console.

“`nav = PileLIFO()`” instancie notre structure.

La boucle “`while True`” : fait tourner le simulateur en continu.

Les conditions “`if/elif`” capturent l'entrée clavier de l'utilisateur pour appeler la bonne méthode algorithmique. Le choix “4” casse la boucle (`break`) pour fermer proprement le programme.

